10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define INPUT_SIZE 200

/* Save shell state */
typedef struct {
    char current_directory[INPUT_SIZE];
    int command_count;
} ShellState;

/* Update current directory */
void updateDirectory(ShellState *state) {

    if (getcwd(state->current_directory,
               sizeof(state->current_directory)) == NULL) {

        perror("getcwd");
    }
}

/* Display current path */
void displayPath(ShellState *state) {

    updateDirectory(state);

    printf("Current Directory: %s\n",
           state->current_directory);
}

/* Cleanup resources */
void cleanup(ShellState *state) {

    printf("\nCleaning up resources...\n");

    /* Save final state */
    updateDirectory(state);

    printf("Final Directory: %s\n",
           state->current_directory);

    printf("Commands processed: %d\n",
           state->command_count);

    printf("Resources cleaned successfully.\n");
}

/* Process exit request */
int processCommand(char *input, ShellState *state) {
```

```
/* Process exit request */
int processCommand(char *input, ShellState *state) {

    /* Exit command */
    if (strcmp(input, "exit") == 0) {

        printf("Exit request received.\n");

        cleanup(state);

        return 0;
    }

    /* pwd command */
    if (strcmp(input, "pwd") == 0) {

        displayPath(state);

        return 1;
    }

    /* help command */
    if (strcmp(input, "help") == 0) {

        printf("\nAvailable Commands:\n");
        printf("  pwd  - Display current directory\n");
        printf("  help - Display available commands\n");
        printf("  exit - Exit the shell\n");

        return 1;
    }

    /* Invalid command */
    printf("Unknown command: %s\n", input);

    return 1;
}

int main() {

    char input[INPUT_SIZE];

    ShellState state;

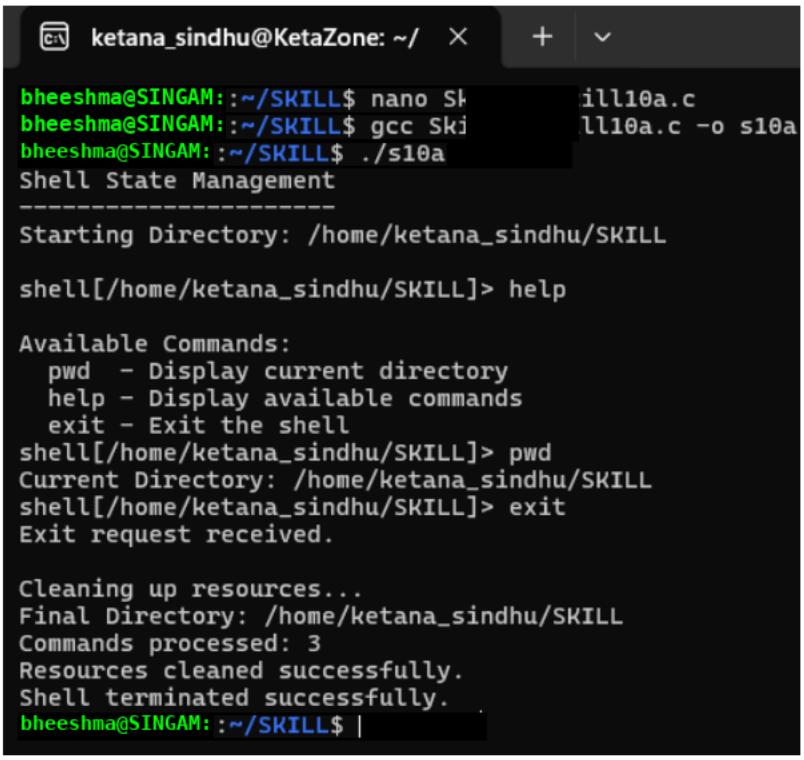
    /* Initialize state */
    state.command_count = 0;

    updateDirectory(&state);

    printf("Shell State Management\n");
    printf("----------------------\n");
```


ketana_sindhu@KetaZone: ~/
GNU nano 8.7.1
ShellState state;
/* Initialize state */
state.command_count = 0;
updateDirectory(&state);
printf("Shell State Management\n");
printf("----------------------\n");
printf("Starting Directory: %s\n\n",
state.current_directory);
while (1) {
printf("shell[%s]> ",
state.current_directory);
fflush(stdout);
if (fgets(input,
sizeof(input),
stdin) == NULL) {
printf("\nInput ended.\n");
cleanup(&state);
break;
}
/* Remove newline */
input[strcspn(input, "\n")] = '\0';
/* Ignore empty input */
if (strlen(input) == 0)
continue;
state.command_count++;
/* Process command */
if (processCommand(input, &state) == 0) {
break;
}
}
printf("Shell terminated successfully.\n");
return 0;
}

Result:


ketana_sindhu@KetaZone: ~/
bheeshma@SINGAM:~/SKILL$ nano Sk ill10a.c
bheeshma@SINGAM:~/SKILL$ gcc Ski ll10a.c -o s10a
bheeshma@SINGAM:~/SKILL$ ./s10a
Shell State Management

Starting Directory: /home/ketana_sindhu/SKILL
shell[/home/ketana_sindhu/SKILL]> help
Available Commands:
pwd - Display current directory
help - Display available commands
exit - Exit the shell
shell[/home/ketana_sindhu/SKILL]> pwd
Current Directory: /home/ketana_sindhu/SKILL
shell[/home/ketana_sindhu/SKILL]> exit
Exit request received.
Cleaning up resources...
Final Directory: /home/ketana_sindhu/SKILL
Commands processed: 3
Resources cleaned successfully.
Shell terminated successfully.
bheeshma@SINGAM:~/SKILL$

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define INPUT_SIZE 200
#define NAME_SIZE 100
#define VALUE_SIZE 200

/* Validate environment variable name */
int validateName(const char *name) {

    if (name[0] == '\0')
        return 0;

    /* First character must be letter or underscore */
    if (!isalpha((unsigned char)name[0]) &&
        name[0] != '_') {
        return 0;
    }

    /* Remaining characters */
    for (int i = 1; name[i] != '\0'; i++) {

        if (!isalnum((unsigned char)name[i]) &&
            name[i] != '_') {
            return 0;
        }
    }

    return 1;
}

/* Process export command */
int processExport(char *input) {

    char *argument = input + 7;    /* Skip "export " */

    /* Remove leading spaces */
    while (*argument == ' ')
        argument++;

    if (*argument == '\0') {
        printf("export: missing variable assignment\n");
        return 1;
    }

    /* Find '=' */
    char *equals = strchr(argument, '=');
```

```
    /* Find '=' */
    char *equals = strchr(argument, '=');

    if (equals == NULL) {
        printf("export: invalid syntax\n");
        printf("Usage: export NAME=value\n");
        return 1;
    }

    /* Extract variable name */
    char name[NAME_SIZE];

    int nameLength = equals - argument;

    if (nameLength <= 0 ||
        nameLength >= NAME_SIZE) {

        printf("export: invalid variable name\n");
        return 1;
    }

    strncpy(name, argument, nameLength);
    name[nameLength] = '\0';

    /* Validate name */
    if (!validateName(name)) {

        printf("export: invalid variable name: %s\n",
               name);

        return 1;
    }

    /* Extract value */
    char value[VALUE_SIZE];

    strncpy(value,
            equals + 1,
            VALUE_SIZE - 1);

    value[VALUE_SIZE - 1] = '\0';

    /* Check whether variable already exists */
    char *oldValue = getenv(name);

    if (oldValue != NULL) {

        printf("Existing variable found.\n");
        printf("Previous value: %s\n", oldValue);
    }
```

```
GNU nano 8.7.1
    /* Update environment variable */
    if (setenv(name, value, 1) != 0) {

        perror("setenv");
        return 1;
    }

    printf("Variable exported successfully.\n");
    printf("%s=%s\n", name, getenv(name));

    return 1;
}

/* Demonstrate variable inherited by child process */
void testChildProcess(const char *name) {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    if (pid == 0) {

        /* Child process */
        printf("\nChild Process:\n");

        char *value = getenv(name);

        if (value != NULL) {
            printf("Child received: %s=%s\n",
                   name, value);
        }
        else {
            printf("Child could not find variable.\n");
        }

        exit(0);
    }

    else {

        /* Parent waits for child */
        waitpid(pid, NULL, 0);

        printf("Child process completed.\n");
    }
}

int main() {
```

```
ketana_sindhu@KetaZone: ~/
GNU nano 8.7.1
        /* Parent waits for child */
        waitpid(pid, NULL, 0);

        printf("Child process completed.\n");
    }
}

int main() {

    char input[INPUT_SIZE];

    printf("Environment Variable Export Shell\n");
    printf("---------------------------------\n");

    printf("Use: export NAME=value\n");
    printf("Use: show NAME\n");
    printf("Use: child NAME\n");
    printf("Use: exit\n\n");

    while (1) {

        printf("shell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL) {
            break;
        }

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty input */
        if (strlen(input) == 0)
            continue;

        /* Exit */
        if (strcmp(input, "exit") == 0) {

            printf("Exiting shell...\n");
            break;
        }

        /* Export */
        if (strncmp(input, "export ", 7) == 0) {

            processExport(input);
        }

        /* Show variable */
```

```c
        }

        /* Export */
        if (strncmp(input, "export ", 7) == 0) {

            processExport(input);
        }

        /* Show variable */
        else if (strncmp(input, "show ", 5) == 0) {

            char *name = input + 5;

            char *value = getenv(name);

            if (value != NULL) {
                printf("%s=%s\n", name, value);
            }
            else {
                printf("Variable '%s' is not defined.\n",
                       name);
            }
        }

        /* Test child process */
        else if (strncmp(input, "child ", 6) == 0) {

            char *name = input + 6;

            testChildProcess(name);
        }

        else {

            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

Result:

